

C# - PHẦN 8.4

QUEUE<T>

Hàng đợi FIFO trong C# và ứng dụng .NET

Mục tiêu: Hiểu nguyên tắc FIFO; sử dụng Enqueue, Dequeue, Peek, TryDequeue, TryPeek; đóng gói Queue trong class; và áp dụng vào hàng đợi xử lý đơn hàng.

- Lý thuyết từ cơ bản đến thực hành
- Ví dụ SupportQueue hoàn chỉnh
- So sánh các collection đã học
- Ứng dụng Queue trong .NET
- Bài tập OrderProcessingQueue - không kèm đáp án

Tài liệu ôn tập cá nhân

1. Queue<T> là gì?

Queue<T> là collection mô phỏng một hàng đợi. Phần tử được đưa vào trước sẽ được lấy ra và xử lý trước.

Nguyên tắc: FIFO - First In, First Out - vào trước, ra trước.

```
Queue<string> customers = new Queue<string>();  
  
customers.Enqueue("An");  
customers.Enqueue("Binh");  
customers.Enqueue("Cuong");
```

Hàng đợi lúc này:

Thứ tự: An -> Binh -> Cuong. An nằm ở đầu hàng đợi vì được thêm trước.

Cú pháp tổng quát

```
Queue<T> queue = new Queue<T>();
```

- T là kiểu dữ liệu của phần tử trong hàng đợi.
- Có thể dùng Queue<string>, Queue<Order>, Queue<SupportTicket>...
- Queue không hỗ trợ truy cập ngẫu nhiên bằng index.

2. Enqueue() - thêm vào cuối hàng đợi

Enqueue() thêm một phần tử vào cuối Queue.

```
customers.Enqueue("An");  
customers.Enqueue("Binh");  
customers.Enqueue("Cuong");
```

Lưu ý: Enqueue() trả về void, không trả về bool.

3. Dequeue() - lấy và xóa phần tử đầu

```
string firstCustomer = customers.Dequeue();  
Console.WriteLine(firstCustomer); // An
```

Sau khi Dequeue(), An bị xóa khỏi Queue; hàng đợi còn Binh -> Cuong.

- Dequeue() lấy phần tử đầu.
- Dequeue() đồng thời xóa phần tử đó.
- Nếu Queue rỗng, Dequeue() ném InvalidOperationException.

4. Peek() - xem phần tử đầu nhưng không xóa

```
Queue<string> customers = new Queue<string>();
customers.Enqueue("An");
customers.Enqueue("Binh");

Console.WriteLine(customers.Peek()); // An
Console.WriteLine(customers.Peek()); // An
```

Method	Lấy phần tử đầu	Xóa phần tử
Peek()	Có	Không
Dequeue()	Có	Có

Queue rỗng và lỗi thường gặp

```
Queue<string> customers = new Queue<string>();
customers.Peek();    // InvalidOperationException
customers.Dequeue(); // InvalidOperationException
```

Cách an toàn: Kiểm tra Count trước hoặc ưu tiên TryPeek()/TryDequeue() khi Queue có thể rỗng.

5. TryDequeue() - lấy và xóa an toàn

```
if (customers.TryDequeue(out string? customer))
{
    Console.WriteLine($"Serving: {customer}");
}
else
{
    Console.WriteLine("The queue is empty.");
}
```

- Queue có dữ liệu: trả về true, gán phần tử đầu vào biến out và xóa phần tử đó.
- Queue rỗng: trả về false, không ném exception.

6. TryPeek() - xem an toàn

```
if (customers.TryPeek(out string? customer))
{
    Console.WriteLine($"Next customer: {customer}");
}
else
{
    Console.WriteLine("The queue is empty.");
}
```

Method	Có xóa?	Khi Queue rỗng
Dequeue()	Có	Ném exception
Peek()	Không	Ném exception
TryDequeue()	Có	Trả false
TryPeek()	Không	Trả false

7. Count, foreach, Contains và Clear

Count

```
Console.WriteLine($"Waiting customers: {customers.Count}");
```

Duyệt bằng foreach

```
foreach (string customer in customers)
{
    Console.WriteLine(customer);
}
```

Quan trọng: foreach chỉ đọc; không xóa phần tử và không làm giảm Count.

Contains()

```
bool exists = customers.Contains("An");
```

Clear()

```
customers.Clear();
Console.WriteLine(customers.Count); // 0
```

8. Không thay đổi Queue bên trong foreach

Không nên gọi Dequeue() trong lúc foreach đang duyệt cùng Queue vì collection bị thay đổi giữa quá trình enumeration.

```
// Không nên
foreach (string customer in customers)
{
    customers.Dequeue();
}
```

Xử lý đến khi Queue rỗng

```
while (customers.TryDequeue(out string? customer))
{
    Console.WriteLine($"Serving: {customer}");
}
```

Khác nhau: foreach dừng khi cần xem hàng đợi mà không thay đổi. while + TryDequeue dừng khi cần xử lý và lấy hết phần tử.

Queue không có index

```
customers[0]; // Không hợp lệ
```

Khi cần truy cập phần tử theo vị trí, List<T> phù hợp hơn.

9. Khởi tạo Queue với dữ liệu ban đầu

```
Queue<string> customers = new Queue<string>(
    new[] { "An", "Binh", "Cuong" });
```

Cũng có thể tạo Queue từ List hoặc collection khác:

```
List<string> names = new List<string>
{
    "An",
    "Binh",
    "Cuong"
};

Queue<string> customers = new Queue<string>(names);
```

Thứ tự: Thứ tự trong collection ban đầu trở thành thứ tự phục vụ của Queue.

10. Encapsulation với Queue<T>

```
private readonly Queue<SupportTicket> _tickets;
```

- private: bên ngoài không thao tác trực tiếp với Queue.
- readonly: field không thể trở sang Queue khác sau khi constructor kết thúc.
- Nội dung Queue vẫn được thay đổi thông qua Enqueue, TryDequeue, Clear... trong class.

11. Vì sao không nên để Queue public?

```
public Queue<SupportTicket> Tickets { get; }
```

Nếu Queue public, bên ngoài có thể bỏ qua quy tắc nghiệp vụ:

```
service.Tickets.Clear();  
service.Tickets.Enqueue(ticket);  
service.Tickets.Dequeue();
```

Thiết kế tốt: Giữ Queue là private readonly và cung cấp các method có kiểm soát.

```
public void AddTicket(SupportTicket ticket)  
public SupportTicket? ViewNextTicket()  
public SupportTicket? ProcessNextTicket()  
public void DisplayWaitingTickets()
```

Nhờ đó class kiểm soát được validation, thứ tự xử lý và thông báo lỗi.

12. Ví dụ SupportTicket

```
public class SupportTicket
{
    public string Id { get; }
    public string CustomerName { get; }
    public string Issue { get; }

    public SupportTicket(string id, string customerName, string issue)
    {
        if (string.IsNullOrEmpty(id))
            throw new ArgumentException(
                "Ticket Id cannot be null or whitespace.", nameof(id));

        if (string.IsNullOrEmpty(customerName))
            throw new ArgumentException(
                "Customer name cannot be null or whitespace.",
                nameof(customerName));

        if (string.IsNullOrEmpty(issue))
            throw new ArgumentException(
                "Issue cannot be null or whitespace.", nameof(issue));

        Id = id.Trim();
        CustomerName = customerName.Trim();
        Issue = issue.Trim();
    }

    public void DisplayInfo()
    {
        Console.WriteLine(
            $"Ticket: {Id}, Customer: {CustomerName}, Issue: {Issue}");
    }
}
```

13. Ví dụ SupportQueue

```
public class SupportQueue
{
    public string Name { get; }
    private readonly Queue<SupportTicket> _tickets;

    public SupportQueue(string name)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException(
                "Queue name cannot be null or whitespace.", nameof(name));

        Name = name.Trim();
        _tickets = new Queue<SupportTicket>();
    }

    public void AddTicket(SupportTicket ticket)
    {
        if (ticket == null)
            throw new ArgumentNullException(
                nameof(ticket), "Ticket cannot be null.");

        _tickets.Enqueue(ticket);
    }

    public SupportTicket? ViewNextTicket()
    {
        if (_tickets.TryPeek(out SupportTicket? ticket))
            return ticket;

        return null;
    }

    public SupportTicket? ProcessNextTicket()
    {
        if (_tickets.TryDequeue(out SupportTicket? ticket))
            return ticket;

        return null;
    }
}
```

14. Hiển thị và đếm ticket đang chờ

```
public int GetWaitingCount()
{
    return _tickets.Count;
}

public void DisplayWaitingTickets()
{
    Console.WriteLine($"Queue: {Name}");

    if (_tickets.Count == 0)
    {
        Console.WriteLine("There are no waiting tickets.");
        return;
    }

    foreach (SupportTicket ticket in _tickets)
    {
        ticket.DisplayInfo();
    }
}
```

Luồng hoạt động

- AddTicket(ticket1), AddTicket(ticket2), AddTicket(ticket3): ticket1 nằm đầu Queue.
- ViewNextTicket(): trả ticket1 nhưng Count không đổi.
- ProcessNextTicket(): trả ticket1 và xóa khỏi Queue.
- Sau đó ticket2 trở thành phần tử đầu.

15. Ứng dụng Queue trong .NET

Ứng dụng	Cách Queue được dùng
Gửi email	Email vào trước được gửi trước
Xử lý đơn hàng	Đơn nhận trước được xử lý trước
Tác vụ nền	Tạo báo cáo, xử lý ảnh, đồng bộ dữ liệu
Hỗ trợ khách hàng	Ticket tạo trước được xử lý trước
Thuật toán BFS	Duyệt theo từng lớp bằng hàng đợi

Lưu ý trong ASP.NET Core

Queue<T> thông thường không được thiết kế để nhiều thread cùng thay đổi đồng thời. Với background worker hoặc xử lý cạnh tranh, .NET thường sử dụng:

- ConcurrentQueue<T>
- Channel<T>
- RabbitMQ, Azure Service Bus, Kafka hoặc message broker khác

Phạm vi hiện tại: Chỉ cần nắm chắc Queue<T>; phần đồng thời và tác vụ nền sẽ học sau khi học async/await.

16. So sánh các collection đã học

Collection	Đặc điểm chính	Thao tác đặc trưng
List<T>	Danh sách có index	Add, Remove, [index]
Dictionary<TKey,TValue>	Tra cứu theo key	TryGetValue, Remove(key)
HashSet<T>	Không chứa phần tử trùng	Add, Contains, Remove
Queue<T>	FIFO - vào trước, ra trước	Enqueue, TryDequeue, TryPeek

Các lỗi thường gặp

1. Gọi Dequeue() hoặc Peek() khi Queue rỗng.
2. Nhầm Peek() là thao tác xóa phần tử.
3. Cố truy cập Queue bằng index.
4. Dùng foreach rồi thay đổi Queue bên trong vòng lặp.
5. Để Queue public và cho phép bên ngoài bỏ qua validation.

Quy tắc nhanh: Có thể rỗng -> dùng TryDequeue/TryPeek. Chắc chắn có phần tử -> có thể dùng Dequeue/Peek.

17. Bài tập 8.4 - Hàng đợi xử lý đơn hàng

Tạo class Order với các thuộc tính:

- Id
- CustomerName
- TotalAmount

Validation: Id và CustomerName không rỗng; TotalAmount > 0. Có method DisplayInfo().

Class OrderProcessingQueue

```
public string Name { get; }  
private readonly Queue<Order> _orders;
```

Method	Yêu cầu
AddOrder(Order order)	Kiểm tra null, dùng Enqueue()
ViewNextOrder()	Dùng TryPeek(), rỗng trả null
ProcessNextOrder()	Dùng TryDequeue(), rỗng trả null
GetWaitingOrderCount()	Trả về Count
DisplayWaitingOrders()	Kiểm tra rỗng, hiển thị Name, duyệt foreach

18. Yêu cầu trong Program.cs

1. Tạo một OrderProcessingQueue.
2. Tạo ít nhất ba đơn hàng.
3. Thêm ba đơn hàng bằng AddOrder().
4. Hiển thị toàn bộ đơn hàng đang chờ.
5. Xem đơn hàng tiếp theo bằng ViewNextOrder().
6. Xác nhận TryPeek() không làm giảm Count.
7. Xử lý một đơn bằng ProcessNextOrder().
8. Kiểm tra số lượng đơn còn lại.
9. Hiển thị lại hàng đợi.
10. Tiếp tục xử lý cho đến khi Queue rỗng.
11. Thử xử lý thêm một lần khi Queue đã rỗng.
12. Hiển thị thông báo phù hợp khi không còn đơn hàng.

Bắt buộc: Queue<Order>, Enqueue(), TryPeek(), TryDequeue(), Count và foreach. Chưa dùng Dequeue()/Peek() trực tiếp trong bài đầu tiên.

Checklist trước khi nộp bài

- Queue được khai báo private readonly.
- Các chuỗi được validation trước khi Trim().
- Method có thể không tìm thấy trả về Order?.
- ViewNextOrder không làm thay đổi Count.
- ProcessNextOrder làm giảm Count đúng một đơn.
- Không thay đổi Queue bên trong foreach.
- Program kiểm tra cả trường hợp Queue có dữ liệu và Queue rỗng.